

Solidity Engineering Standard — v2.0

Owner: Solidity team · **Approved:** Praveen (CTO)

Applies to: all production EVM contracts, their tests, scripts, deployment configuration and operational controls

Supersedes: Solidity Engineering Standard & Protocol Guidelines v1.0

How to read this document

Part A is the ten rules that prevent losses. Each has vulnerable code and fixed code. Every Solidity developer reads Part A and can recite it.

Part B is the full standard — the complete rule set, for reference during design and review.

Part C is the review checklist. Part D is the PR template.

Repository-specific material — a project’s analysis baseline, its enforcement phase, its deployed contracts and its open findings — lives in that repository’s `docs/`, not here. This document is portable across projects.

A rule in Part A also appears in Part B. Part A exists because a rule nobody reads is not a control.

Rule levels

Level	Meaning
[BLOCKER]	A merge or release cannot proceed. An exception needs a named approver, a compensating control, and an expiry date.
[TOOL]	A linter, compiler setting, test or CI step enforces it.
[REVIEW]	A human must reason about it.
[CONDITIONAL]	Applies when the named feature or risk exists.

[BLOCKER] is severity and combines with the others: a [BLOCKER] [TOOL] rule is both enforced by CI and release-stopping.

Style findings never outweigh correctness, security, economic safety or upgrade safety.

Toolchain

- **Hardhat** for deployment, scripting and verification.
- **Foundry** for testing. Fuzz and invariant testing are the two highest-value test types in this stack and Foundry is where they live. New projects start with both; existing projects add Foundry for tests without migrating deployment.
- **Slither** on every PR, **Mythril** on a nightly schedule. Mythril is kept off the PR path because it costs far more CI time than a pull request can absorb.

PART A — The ten rules that prevent losses

Ordered by how often they appear in real EVM losses and in static-analysis findings on production code — not by theoretical severity.

A1. Lock every implementation contract

[BLOCKER] **An upgradeable implementation disables its own initializers in its constructor.**

Without it, anyone can call `initialize()` on the implementation directly, become its owner, and — depending on the pattern — `upgradeTo` a contract containing `selfdestruct`, bricking every proxy that points at it. This is the Parity multisig failure. It requires no user to have approved anything.

```
// x VULNERABLE – the implementation is uninitialised and anyone can claim it
contract ExchangeV4 is UUPSUpgradeable, OwnableUpgradeable {
    function initialize(address admin) public initializer {
        __Ownable_init(admin);
    }
}
```

```
}
}
```

```
// ✓ FIXED
contract ExchangeV4 is UUPSUpgradeable, OwnableUpgradeable {
    /// @custom:oz-upgrades-unsafe-allow constructor
    constructor() {
        _disableInitializers(); // the implementation can never be initialised
    }

    function initialize(address admin) public initializer {
        __Ownable_init(admin);
    }

    function _authorizeUpgrade(address) internal override onlyOwner {}
}
```

[BLOCKER] `_authorizeUpgrade` is implemented and restricted. A UUPS contract with an empty, unguarded `_authorizeUpgrade` is upgradeable by anyone.

[TOOL] A deployment test asserts `initialize()` reverts on the implementation address.

A2. Check the return value of every token transfer

[BLOCKER] Use `SafeERC20`. Never call `transfer`, `transferFrom` or `approve` on an ERC-20 directly.

Some tokens return `false` instead of reverting. Some return nothing at all, which makes a bare call succeed against a non-contract address. A transfer that silently fails while the contract records it as succeeded is a direct loss.

```
// ✗ VULNERABLE – a token returning false is treated as success
token.transfer(recipient, amount);
```

```
// ✓ FIXED
using SafeERC20 for IERC20;

token.safeTransfer(recipient, amount);
```

[BLOCKER] Account using observed balance deltas where fee-on-transfer tokens are supported, otherwise reject them explicitly and document the restriction. A fee-on-transfer token credited at the requested amount rather than the received amount drains the contract over time.

```
uint256 before = token.balanceOf(address(this));
token.safeTransferFrom(msg.sender, address(this), amount);
uint256 received = token.balanceOf(address(this)) - before; // credit this, not `amount`
```

A3. Never let the caller choose the `from` address

[BLOCKER] A `transferFrom` whose `from` is caller-supplied lets anyone move tokens from anyone who has approved the contract.

This is a one-line exploit that drains every user who has ever granted an allowance.

```
// ✗ VULNERABLE – `from` comes from calldata
function deposit(address from, uint256 amount) external {
    token.safeTransferFrom(from, address(this), amount);
}
```

```
// ✓ FIXED – the source is always the caller
function deposit(uint256 amount) external {
    token.safeTransferFrom(msg.sender, address(this), amount);
}
```

Where a third party genuinely must move funds on a user's behalf, the authority comes from a signed permit with a nonce and deadline (A8) — never from a parameter.

A4. Checks, effects, interactions — and a guard

[BLOCKER] **State is updated before any external call.** Add `nonReentrant` where cross-function or callback reentrancy remains possible.

Any external call — a token transfer, an ETH send, a hook, a receiver callback, an oracle read — can re-enter.

```
// ✗ VULNERABLE – balance is cleared after the call
function withdraw() external {
    uint256 amount = balances[msg.sender];
    (bool ok, ) = msg.sender.call{value: amount}("");
    require(ok);
    balances[msg.sender] = 0;           // too late
}
```

```
// ✓ FIXED – checks, effects, then interactions
function withdraw() external nonReentrant {
    uint256 amount = balances[msg.sender];    // check
    require(amount > 0, NothingToWithdraw());
    balances[msg.sender] = 0;                 // effect
    (bool ok, ) = msg.sender.call{value: amount}(""); // interaction
    require(ok, TransferFailed());
}
```

[REVIEW] **Read-only reentrancy counts.** A view function consumed by another protocol can be called mid-transaction while our state is inconsistent.

[BLOCKER] **Prefer pull over push** where a failing receiver could block global progress. One recipient that reverts must not freeze everyone else's settlement.

A5. Authorisation is a role, never an address comparison or `tx.origin`

[BLOCKER] **Never authorise with `tx.origin`**. It authorises whatever contract the user was tricked into calling.

[BLOCKER] **Every privileged function has the minimum required role and a stated trust rationale.** Separate upgrade, pause, configuration, treasury, settlement and oracle powers when their blast radius differs — one `onlyOwner` covering all of them means one key loss is total.

[BLOCKER] **Ownership and critical admin transfers use a two-step flow.** A single-step transfer to a mistyped address is unrecoverable.

```
// ✗ one role, one key, total authority
contract Vault is Ownable {
    function setOracle(address o) external onlyOwner { ... }
    function pause() external onlyOwner { ... }
    function sweep(address to) external onlyOwner { ... }
}
```

```
// ✓ separated by blast radius, two-step ownership
contract Vault is AccessControl, Ownable2Step {
    bytes32 public constant PAUSER_ROLE = keccak256("PAUSER_ROLE");
    bytes32 public constant ORACLE_ADMIN_ROLE = keccak256("ORACLE_ADMIN_ROLE");
    bytes32 public constant TREASURY_ROLE = keccak256("TREASURY_ROLE");

    function pause() external onlyRole(PAUSER_ROLE) { ... }
    function setOracle(address o) external onlyRole(ORACLE_ADMIN_ROLE) { ... }
    function sweep(address to) external onlyRole(TREASURY_ROLE) { ... }
}
```

[TOOL] **Tests enumerate every role and prove an unauthorised caller reverts on every privileged entry point.** This is mechanical and should exist for every contract.

[BLOCKER] **Route guards, frontend restrictions, `private` functions and obscure calldata are never treated as authorisation.**

A6. Rounding has a direction, and it is always against the user

[BLOCKER] **Every conversion and share formula defines its rounding direction per path. A deposit/mint and withdraw/redeem pair cannot be rounded in a way that creates free value.**

The classic failure: rounding in the user's favour on both entry and exit lets an attacker cycle and extract the dust.

```
// ✗ both directions round down – the pair leaks value
function toShares(uint256 assets) public view returns (uint256) {
    return assets * totalShares / totalAssets;
}
```

```
function toAssets(uint256 shares) public view returns (uint256) {
    return shares * totalAssets / totalShares;
}
```

```
// ✓ rounding favours the protocol on both sides
function depositToShares(uint256 assets) public view returns (uint256) {
    return Math.mulDiv(assets, totalShares, totalAssets, Math.Rounding.Floor);
}
function redeemToAssets(uint256 shares) public view returns (uint256) {
    return Math.mulDiv(shares, totalAssets, totalShares, Math.Rounding.Floor);
}
function withdrawToShares(uint256 assets) public view returns (uint256) {
    return Math.mulDiv(assets, totalShares, totalAssets, Math.Rounding.Ceil);
}
```

[BLOCKER] **Use full-precision `mulDiv`** where intermediate multiplication can overflow or truncation is material. `a * b / c` loses precision that `mulDiv` does not.

[BLOCKER] **unchecked blocks carry a proven bound and focused tests.** `unchecked` is never used merely for style or gas feel.

[BLOCKER] **Every amount has a defined unit and decimal scale at every contract boundary**, documented in NatSpec. Mixing 6-decimal and 18-decimal tokens without explicit normalisation is a loss, not a bug.

A7. An oracle read is validated, or it is a trusted attacker

[BLOCKER] **Validate freshness, decimals, sign and range, round completeness, and sequencer status where the chain has one.**

[BLOCKER] **A spot price from a manipulable pool is never a trusted oracle** without an explicit manipulation-cost analysis. A flash loan makes spot price free to move.

```
// ✗ VULNERABLE – stale, negative or incomplete data accepted
(, int256 price, , , ) = feed.latestRoundData();
uint256 value = amount * uint256(price);
```

```
// ✓ FIXED
(uint80 roundId, int256 price, , uint256 updatedAt, uint80 answeredInRound) = feed.latestRoundData();
if (price <= 0) revert InvalidPrice();
if (answeredInRound < roundId) revert StaleRound();
if (block.timestamp - updatedAt > MAX_STALENESS) revert PriceTooOld();
uint256 value = Math.mulDiv(amount, uint256(price), 10 ** feed.decimals());
```

[BLOCKER] **Stale, zero, negative, extreme, divergent or unavailable data has an explicit fail-closed, fallback or pause behaviour.** Defaulting to the last known price is a decision, and it must be a deliberate one.

[BLOCKER] **User trades carry amount bounds, slippage protection and deadlines** wherever price can move between submission and execution.

A8. Every signature binds everything, and every nonce is single-use

[BLOCKER] **A typed signature binds chain id, domain, verifying contract, action, every security-relevant parameter, a nonce and a deadline.**

[BLOCKER] **Nonces are single-use in the intended scope and invalidated before or atomically with the action.**

```
// ✗ VULNERABLE – replayable on another chain, after any delay, by any relayer
bytes32 digest = keccak256(abi.encodePacked(to, amount));
address signer = ECDSA.recover(digest, signature);
require(signer == owner);
```

```
// ✓ FIXED – EIP-712, bound and bounded
bytes32 constant CLAIM_TYPEHASH =
    keccak256("Claim(address to,uint256 amount,uint256 nonce,uint256 deadline)");

function claim(address to, uint256 amount, uint256 deadline, bytes calldata sig) external {
    if (block.timestamp > deadline) revert SignatureExpired();
    uint256 nonce = nonces[to]++; // consumed before the action
    bytes32 digest = _hashTypedDataV4(
        keccak256(abi.encode(CLAIM_TYPEHASH, to, amount, nonce, deadline))
    );
    require(verifySignature(digest, sig, to));
}
```

```

    );
    if (ECDSA.recover(digest, sig) != signerAddress) revert InvalidSignature();
    _transfer(to, amount);
}

```

[BLOCKER] A relay cannot substitute recipient, amount, fee, token, market, calldata or deadline. Anything omitted from the signed payload is something the relay chooses.

[BLOCKER] Signature recovery rejects invalid and malleable signatures — use a vetted library, never raw `ecrecover`.

A9. No unbounded loop on a path required for withdrawal or recovery

[BLOCKER] No user-controlled unbounded iteration on a liveness-critical path — withdrawal, settlement, liquidation or recovery.

An array a user can grow becomes a denial of service that locks funds permanently once iterating it exceeds the block gas limit.

```

// ✗ VULNERABLE — anyone can add holders until withdrawal cannot be executed
function distribute() external {
    for (uint256 i = 0; i < holders.length; i++) {
        token.safeTransfer(holders[i], shareOf(holders[i]));
    }
}

```

```

// ✓ FIXED — pull settlement; each account claims its own
mapping(address => uint256) public claimable;

function claim() external nonReentrant {
    uint256 amount = claimable[msg.sender];
    if (amount == 0) revert NothingToClaim();
    claimable[msg.sender] = 0;
    token.safeTransfer(msg.sender, amount);
}

```

[REVIEW] Batch operations isolate failures, or document atomic behaviour and a tested maximum safe batch size.

A10. Nothing on chain is private, and nothing on chain is random

[BLOCKER] Sensitive data is never assumed private on chain, even in a `private` variable or passed only as calldata. `private` restricts Solidity access, not storage reads.

[BLOCKER] `block.timestamp`, `blockhash`, `prevrandao`, transaction ordering and private state are never used as secure randomness without an appropriate protocol. Validators and searchers influence all of them.

Where randomness has value attached, use a commit-reveal scheme or a verifiable randomness service, and document the trust assumption.

PART B — The standard

B1. Scope and change discipline

- **[REVIEW]** Every change maps to an explicit requirement, threat, bug or measurable optimisation.
- **[REVIEW]** A PR contains the smallest coherent change. Unrelated refactors, compiler upgrades, dependency upgrades and formatting sweeps are separate.
- **[BLOCKER]** Before changing financial logic, state custody boundaries, trusted actors, units, formulas, rounding, state transitions and invariants are stated in the PR.
- **[BLOCKER]** A bug fix includes a test that fails on the vulnerable revision and passes with the fix.
- **[REVIEW]** Public ABI, emitted events, errors, storage, the permission graph and off-chain integrations are compatibility surfaces.
- **[TOOL]** Generated artifacts, caches, secrets, local keystores, broadcasts containing sensitive data and environment files are excluded from version control.

B2. Toolchain and reproducibility

- [TOOL] Pin the Solidity compiler, framework, package manager, direct dependencies, target EVM version, optimizer runs and via-IR choice.
- [TOOL] Commit exactly one lockfile; use frozen or immutable installs in CI.
- [BLOCKER] **Compiler warnings are zero, or individually documented and approved.**
- [TOOL] Build output is reproducible from the source commit and lockfile.
- [REVIEW] Check the pinned compiler against the Solidity known-bugs list before release.
- [REVIEW] A compiler, EVM target, optimizer or via-IR change gets the same scrutiny as a code change: full tests, bytecode diff, gas diff, storage diff and deployment rehearsal.
- [CONDITIONAL] Multi-chain builds document per-chain opcode, precompile, gas, finality and size-limit differences.

B3. Naming, files and layout

- [TOOL] A Solidity file and its principal contract share a PascalCase name: `MarketFactory.sol` / `MarketFactory`.
- [TOOL] Contracts, libraries, structs, events and custom errors `PascalCase`; functions and variables `mixedCase`; constants and immutables `UPPER_SNAKE_CASE`.
- [REVIEW] Boolean names read as assertions: `isSettled`, `hasRole`, `canRedeem`.
- [REVIEW] **Names encode unit or domain wherever ambiguity can lose funds:** `collateralAmount`, `feeBps`, `deadlineTimestamp`, `priceWad`.
- [TOOL] Interfaces begin with `I`. Abstract base contracts use an `Abstract` suffix only when it improves discovery.
- [TOOL] State visibility is explicit. Integer widths are explicit where storage layout, ABI compatibility, packing or bounds matter.
- [TOOL] Source order: licence, pragma, imports, declarations, contract, types, state, events, errors, modifiers, constructor/initializer, external/public functions, internal/private functions.
- [TOOL] Formatting is owned by the configured formatter. Review does not debate formatting.
- [REVIEW] One major contract or library per file.

B4. Repository architecture

src/ (or contracts/)	production contracts grouped by protocol capability, not by Solidity keyword
interfaces/	stable integration surfaces
libraries/	focused stateless or storage libraries
mocks/	test-only adversarial and dependency doubles
test/	unit, fuzz, invariant, fork, integration and upgrade tests
script/ or deploy/	deterministic deployment, configuration, verification and migration
docs/	architecture, invariants, threat model, roles, formulas and runbooks

- [REVIEW] Contracts are cohesive and small enough to audit. **Split by trust boundary or state responsibility, not by line count.**
- [REVIEW] Dependency direction is explicit. Core accounting does not import deployment, mocks, UI concerns or protocol adapters.
- [REVIEW] Interfaces expose only required behaviour. No public state or functions added only for tests.
- [BLOCKER] **A modular design must not create hidden `delegatecall`, callback or cross-contract invariant failures.**

B5. Documentation and specifications

- [TOOL] Every production file has an SPDX identifier and a compatible pragma.
- [REVIEW] Public and external interfaces carry NatSpec for purpose, parameters, returns, emitted effects, reverts, **units**, and security-sensitive assumptions.
- [BLOCKER] **Non-obvious financial formulas document derivation, units, rounding direction, valid domain, precision bound and examples.**
- [BLOCKER] **The protocol documents assets under custody, liabilities, solvency condition, privileged actions, emergency controls and upgrade authority.**
- [REVIEW] Comments explain why, invariants and hazards — not a literal restatement of code.
- [REVIEW] README and runbooks cover build, test, environment variables, local fork, deployment, verification, role transfer, pause, upgrade and incident response.

B6. Types, state and state machines

- [REVIEW] Use enums or explicit state machines for lifecycle phases. Do not infer phase from several loosely related booleans.
- [BLOCKER] **Every state transition defines caller, preconditions, effects, interactions, events, replay behaviour and recovery path.**

- [REVIEW] Custom value types where they prevent mixing materially different quantities or identifiers.
- [BLOCKER] **Storage defaults are safe. Zero must not silently mean both uninitialised and a valid value.**
- [REVIEW] Pack storage only when the measured saving justifies readability and upgrade-layout constraints.
- [BLOCKER] **Deleting an array or struct containing mappings does not clear those mappings.**
- [REVIEW] Mark immutable configuration `immutable` where the proxy architecture permits; otherwise validate and protect configuration setters.

B7. Access control and governance

See A5 for the blockers. Additionally:

- [REVIEW] Production authority uses an appropriate multisig and, for high-impact delayed actions, a timelock.
- [BLOCKER] **Initializers, reinitializers, implementations, factories, clones and deployers cannot be taken over.**
- [REVIEW] Document the maximum damage, detection mechanism, revocation path and recovery plan for each role.

B8. External calls and reentrancy

See A4 for the blockers. Additionally:

- [BLOCKER] **Low-level `call`, `staticcall` and `delegatecall` results and returndata are validated.**
- [BLOCKER] **Arbitrary-call executors constrain target, selector, value, approvals and replay** according to an explicit capability model.

B9. Assets and token compatibility

See A2 for the blockers. Additionally:

- [REVIEW] Define support for rebasing, ERC-777 hooks, ERC-4626 shares, permit variants, blacklists, paused tokens, transfer caps and unusual decimals.
- [BLOCKER] **Native currency receive/fallback behaviour and forced-balance assumptions are explicit.** A contract can be sent ETH it did not expect.
- [BLOCKER] **Internal liabilities reconcile with actual balances** under donation, direct transfer, dust and rounding scenarios.
- [REVIEW] Approval lifecycle minimises standing allowance. Consider reset-to-zero behaviour and permit replay.
- [BLOCKER] **Rescue functions cannot extract user liabilities, collateral, claimable rewards or protocol-owned accounting assets.**

B10. Accounting, math, fees and rounding

See A6 for the blockers. Additionally:

- [BLOCKER] **Tests cover zero, one unit, minimum, maximum, exact division, one below and above a boundary, repeated operations, skewed reserves and decimal mismatch.**
- [BLOCKER] **Fee order, fee base, recipients, caps, compounding, refunds and fee-on-fee behaviour are explicit.**
- [BLOCKER] **Preview/quote functions and execution use one source of truth, or have a documented maximum deviation.**
- [BLOCKER] **Solvency, conservation and no-free-profit properties are expressed as executable invariants.**
- [REVIEW] Dust ownership and sweep behaviour are documented and cannot be gamed through repetition.

B11. Oracles, pricing and MEV

See A7 for the blockers. Additionally:

- [REVIEW] Manipulation cost is assessed against extractable value and the exact averaging window and liquidity source.
- [REVIEW] Analyse front-running, back-running, sandwiching, liquidation ordering, JIT liquidity, griefing and transaction censorship.
- [CONDITIONAL] Commit-reveal, batch auctions, private order flow or delayed execution where ordering changes the outcome materially.

B12. Signatures, permits and account abstraction

See A8 for the blockers. Additionally:

- [REVIEW] Support contract wallets through EIP-1271 where the user model requires it.
- [REVIEW] Define behaviour across forks, chain-ID changes, upgrades, proxy addresses, batching, relayers and smart accounts.

B13. Upgradeability and proxies

See A1 for the implementation-lock blocker. Additionally:

- [BLOCKER] **The proxy pattern is documented and matches the implementation and deployment tooling.**
- [BLOCKER] **Initializers are idempotence-protected, called exactly once, and initialise all inherited contracts in a valid order.**
- [BLOCKER] **Storage layout compatibility is machine-checked for every upgrade.** Variables are not reordered, removed or type-changed in place.
- [REVIEW] Prefer namespaced storage or the repository's established safe layout strategy. Storage gaps are maintained according to the chosen versioned pattern.
- [BLOCKER] **UUPS implementations validate upgrade authorisation and compatibility; transparent proxies preserve admin separation; beacons protect fleet-wide blast radius.**
- [BLOCKER] **Upgrade tests preserve balances, roles, configuration, queues, nonces, claims and active lifecycle state.**
- [REVIEW] Every upgrade includes migration steps, rehearsal, bytecode and storage diff, rollback or forward-fix plan, monitoring, and timelock/multisig execution data.
- [BLOCKER] **Delegatecall targets and diamond facets/selectors cannot collide or become unauthorised.**

B14. Cross-chain, bridges and L2

- [BLOCKER] **Messages bind source chain, destination chain, sender, receiver, payload, nonce and version.**
- [BLOCKER] **Replay protection is scoped correctly and consumed atomically.**
- [REVIEW] Finality, reorgs, duplicate delivery, out-of-order delivery, delayed delivery, relayer censorship and destination failure are modelled.
- [BLOCKER] **Mint/unlock/burn/lock accounting maintains global supply and cannot process the same transfer twice.**
- [REVIEW] Rate limits, caps, pause domains, validator and quorum assumptions, and compromised-message recovery limit blast radius.
- [CONDITIONAL] L2 logic accounts for sequencer downtime, L1 data availability, address aliasing, retryable messages and chain-specific gas semantics.

B15. Events, errors and off-chain observability

- [TOOL] Custom errors for stable failure categories unless compatibility requires strings.
- [REVIEW] Events allow indexers to reconstruct meaningful state transitions, asset movement, configuration, role, pause, settlement and upgrade changes.
- [REVIEW] Index fields used for filtering; avoid indexing values never queried.
- [BLOCKER] **Event data and state cannot disagree about amounts, recipients, IDs or status.**
- [REVIEW] Off-chain services define confirmation depth, reorg rollback, idempotency, missed-event recovery and backfill. (*Our Node standard covers the backend half of this.*)

B16. Gas, bytecode and scalability

- [TOOL] CI records runtime and init bytecode size plus deployment and critical-path gas against a reviewed baseline.
- [BLOCKER] **Bytecode remains below applicable chain limits with explicit safety headroom.**
- [REVIEW] Optimise measured hot paths, not theoretical micro-costs. Preserve clarity unless savings are material.
- [BLOCKER] **An optimisation cannot weaken validation, precision, upgrade safety or invariant readability.**
- [REVIEW] Assembly documents memory ownership, scratch space, ABI layout, returndata, revert propagation and compiler assumptions. Focused equivalence tests are mandatory.

B17. Testing strategy

- [BLOCKER] **Critical paths have positive, negative, boundary, authorisation, event and state-transition tests.**
- [BLOCKER] **Every material accounting formula has an independent reference model or differential test.**
- [TOOL] Fuzz tests define realistic bounds and reject excessive filtering that hides the input domain.
- [BLOCKER] **Stateful invariants cover conservation, solvency, supply, caps, replay, single claims, valid lifecycle states and permission safety.**
- [REVIEW] Handler-based invariant tests model realistic actors, time, prices, callbacks, donations, reentrancy and operation ordering.
- [CONDITIONAL] Fork tests verify live token, oracle, DEX, proxy and protocol integration at a pinned block.

- [CONDITIONAL] Upgrade tests compare layouts and preserve live-like state.
- [REVIEW] Adversarial mocks include reentrant, reverting, gas-griefing, non-returning, false-returning, fee-on-transfer, rebasing and callback tokens.
- [TOOL] Test seeds and failing sequences are reproducible; failures are minimised into regression tests.
- [REVIEW] **Coverage is a gap detector. High coverage with weak assertions is not a pass.**

B18. Security analysis and audit readiness

- [TOOL] Formatting, compilation, unit tests, fuzz tests, invariant tests, static analysis, size checks and storage-layout checks run in CI.
- [REVIEW] **Static-analysis findings are triaged. Suppressions are narrow, documented and reviewed** — never a blanket filter.

Suppress the narrowest thing possible, in code, with a reason:

```
// slither-disable-next-line reentrancy-eth
someFlaggedCall();
```

- [BLOCKER] **Every suppression is recorded** in the PR or a linked security issue with the finding ID, severity, contract, justification, approver and remediation reference. **Approval for a high-severity suppression cannot come from the author of the change alone.**
- [REVIEW] **A broad path filter is not a suppression strategy.** Analysers drop an entire finding when any element of its trace touches a filtered path, so filtering dependency or test directories routinely hides first-party findings whose traces pass through a library. Prefer a dependency-exclusion setting that keeps first-party results, and measure what a filter removes before adopting it.
- [BLOCKER] **A project gates on new findings against an approved, version-controlled baseline — never on the absolute count.** The baseline is approved by a named owner after triage, not generated and accepted in the same change. **Regenerating the baseline to clear a new finding is a policy violation, not an override;** the baseline is version-controlled precisely so that doing so shows up in review.
- [REVIEW] **Enforcement is phased, and the phase is recorded in the repository.** Report-only while the initial findings are triaged; then block on new high-severity findings against an approved baseline; then make the check required in branch protection; then tighten the severity threshold for custody, upgradeability and token-accounting contracts. Advancing a phase is a reviewed configuration change, and that change is the audit record.
- [CONDITIONAL] Symbolic execution and SMT/model checking target arithmetic, reachability, authorisation and assertions where bounded analysis is useful.
- [CONDITIONAL] Property fuzzing supplements framework-native invariants for high-value protocols.
- [BLOCKER] **Critical and high findings are fixed, or governed by an explicit release exception with compensating controls.**
- [REVIEW] Pre-audit material includes scope and commit, architecture, trust model, roles, invariants, formulas, deployment topology, known issues, test instructions and out-of-scope dependencies.
- [BLOCKER] **No report says audit tools proved safety.** Tool limitations and unverified assumptions are stated.

B19. Deployment, verification and configuration

- [BLOCKER] **Deployment scripts are deterministic, idempotent where practical, chain-ID checked, and fail on missing or unexpected configuration.**
- [BLOCKER] **Scripts verify deployed code, proxy implementation/admin/ beacon slots, initializer arguments, roles, ownership, caps, fees and oracle addresses.**
- [REVIEW] CREATE2 salts, predicted addresses, nonce assumptions and front-running implications are documented when used.
- [BLOCKER] **Deployer privileges are transferred or revoked according to plan and verified on chain.**
- [TOOL] Source verification and bytecode matching are part of the release process.
- [REVIEW] A dry run or fork rehearsal produces a reviewed transaction manifest before mainnet execution.
- [BLOCKER] **Production execution uses approved multisig/timelock calldata. Private keys are never embedded in scripts, history, CI logs or shell arguments.**
- [REVIEW] Post-deployment smoke tests exercise read paths and safe low-risk transactions.
- [BLOCKER] **No mainnet deployment without either a completed third-party audit, or a written client acknowledgment that an audit was declined and the risk accepted.** This is raised at proposal stage, not at deploy time — an audit introduced as a surprise at deployment is a conversation that is lost before it starts.

B20. Monitoring and incident response

- [BLOCKER] **High-value systems define monitors for upgrades, role changes, pauses, oracle anomalies, reserve/liability divergence, large outflows, failed keepers and invariant drift.**

- [REVIEW] Alerts have an owner, severity, threshold, deduplication, escalation and runbook. *(Same alerting discipline as section 4.4 of the Node and Python standards.)*
- [REVIEW] Emergency controls minimise harm while preserving withdrawals when safely possible.
- [BLOCKER] **Pause/unpause authority, scope and invariants are tested. Pausing cannot silently strand assets without a recovery design.**
- [REVIEW] Incident plans cover communication, evidence preservation, key rotation, permission revocation, upgrade/patch, recovery and postmortem.
- [REVIEW] Recovery time and maximum tolerable loss are declared for external dependencies and operational services.

B21. Dependencies and supply chain

- [REVIEW] Every new dependency states purpose, alternative considered, audit and maintenance status, licence, pinned version, bytecode and gas impact, and trust introduced.
- [TOOL] Dependency advisories and licence policy run in CI.
- [BLOCKER] **Imported code is compiled from the reviewed version; remappings cannot silently resolve another copy.**
- [BLOCKER] **Do not copy unaudited snippets for cryptography, fixed-point math, signatures, proxies, Merkle proofs or token transfers.**

B22. PR and merge rules

- [BLOCKER] **Merge is blocked by:** failing build or tests, compiler warnings, unresolved blocking review comments, reduced required coverage, size-limit failure, layout incompatibility, untriaged high-severity analysis findings, or missing deployment/config evidence.
- [REVIEW] **At least one qualified smart-contract reviewer approves. Two are required for custody, accounting, authorisation, signature, oracle, cross-chain, proxy or settlement changes.**
- [REVIEW] Authors do not self-approve production changes or bypass protected branches.
- [REVIEW] Security-sensitive changes include an adversarial review distinct from the implementation review.
- [REVIEW] A PR targets the smallest coherent change — the same 400-line guidance as our other stacks, with the understanding that contract diffs are denser and a smaller target is often right.

On the reviewer requirement. Where a project does not have a second reviewer with contract expertise, the two-reviewer rule for custody, accounting, authorisation, signature, oracle, cross-chain, proxy and settlement changes is **met by an external reviewer or an audit engagement, not waived**. In every other stack a missing second review is a quality gap. Here it is a financial one, because the artifact is immutable and holds value from the moment it is deployed.

B23. Maintainability and complexity

- [TOOL] No dead code, commented-out blocks, unused imports, debug events, focused or skipped tests without a ticket, or unresolved security TODOs.
- [REVIEW] Complex functions split at coherent invariant boundaries. **Line count is a review trigger, not a safety proof.**
- [REVIEW] Modifier logic remains simple; substantial state changes live in functions where execution order is visible.
- [REVIEW] Inheritance is shallow and intentional. Confirm linearisation and overridden behaviour.
- [REVIEW] Duplicate accounting logic has one authoritative implementation before divergence becomes possible.
- [REVIEW] Public surface area is minimised; each external selector increases security and compatibility cost.

B24. Prohibited patterns

- [BLOCKER] `tx.origin` authorisation.
- [BLOCKER] Unprotected initializer, upgrade, mint, burn, sweep, settlement, oracle or arbitrary-call path.
- [BLOCKER] Silent failure of low-level calls or token transfers.
- [BLOCKER] User-controlled unbounded iteration on a liveness-critical path.
- [BLOCKER] `block.timestamp`, `blockhash`, `prevrandao`, transaction order or private state used as secure randomness.
- [BLOCKER] Spot price from a manipulable pool used as a trusted oracle without manipulation analysis.
- [BLOCKER] Floating-point assumptions, JavaScript number parity, or implicit decimal conversion in financial logic.
- [BLOCKER] Storage reordering or type replacement in an upgrade.

- [BLOCKER] `selfdestruct`, `delegatecall`, assembly, transient storage or experimental opcodes without documented necessity, target-chain analysis and tests.
- [REVIEW] `transfer` / `send` as a universal native-currency strategy — choose call/pull semantics based on failure and reentrancy requirements.
- [REVIEW] Generic `Manager`, `Helper`, `Utils`, `Data`, `Info`, `Temp` or `Misc` abstractions that conceal responsibility.
- [REVIEW] Catching or converting a real failure into a successful empty or zero result without explicit product semantics.

PART C — Review checklist

Mark each item pass, fail, not applicable or not measured, **with evidence**.

Scope and compatibility 1. Does the change implement only the stated requirement? 2. Are compiler, EVM, optimizer, via-IR, dependencies, remappings and target chains unchanged unless explicitly required? 3. Are ABI, selectors, events, errors, storage, CREATE2 addresses and integration assumptions compatible? 4. Are all changed trust assumptions and assets at risk documented?

Authorisation and lifecycle 5. Is every privileged path protected by the correct role — including initializer, reinitializer, upgrade, pause, configuration, oracle, mint, burn, sweep, settlement and arbitrary execution? 6. Can any role escalate itself, bypass delay, seize user assets, or permanently freeze the system? 7. Are ownership and role transfers safely confirmed and deployer permissions removed? 8. Are lifecycle transitions complete, exclusive, replay-safe, observable and recoverable?

Accounting and math 9. Do assets, liabilities, shares, supply, fees, rewards and reserves reconcile after every path? 10. Are units, decimal scales, bounds, fee bases and rounding direction explicit? 11. Can zero, dust, donation, first deposit, repeated rounding, skewed reserves or maximum values create free value or insolvency? 12. Do quote/preview and execution paths share the same math and fee assumptions? 13. Is every `unchecked` block proven safe?

External interactions 14. Can any call, token, hook, receiver, oracle, DEX, bridge or callback reenter or fail unexpectedly? 15. Are low-level return values and returndata validated? 16. Can a failed receiver or malicious token block global progress? 17. Are loops and batches bounded for worst-case on-chain execution? 18. Are non-standard tokens supported correctly or rejected explicitly?

Market, oracle and ordering 19. Are freshness, decimals, sign/range, sequencer, deviation and fallback checks correct? 20. Can a flash loan or short-lived liquidity manipulate the consumed price? 21. Are slippage, deadlines, minimum output and maximum input enforced? 22. Can front-running, sandwiching, grieving, partial fills or cancellation ordering violate intent?

Signatures and cross-chain 23. Does the signed payload bind every security-relevant field, domain, chain, verifying contract, nonce and deadline? 24. Is replay impossible across users, actions, contracts, chains and upgrades? 25. Are EIP-1271 and relayer substitution handled when relevant? 26. Do bridge messages remain safe under duplicate, delayed, reordered or failed delivery, and does lock/mint/burn/unlock preserve global supply?

Upgrade safety 27. Is the implementation locked and every initializer protected? 28. Is storage layout machine-checked and compatible? 29. Are inherited initializers ordered correctly? 30. Are upgrade authorisation, admin separation, selector collision and implementation compatibility tested? 31. Does the migration preserve all live balances, roles, queues, claims, nonces, configuration and lifecycle state? 32. Is there a rehearsed execution, monitoring and recovery plan?

Tests and analysis 33. Does every changed behaviour have positive, negative, boundary, authorisation, event and regression evidence? 34. Are economic formulas compared with an independent model? 35. Do stateful invariants cover solvency, conservation, replay, caps, lifecycle and permissions? 36. Are adversarial callbacks, tokens, receivers, time, prices and actors modelled? 37. Are static-analysis findings triaged rather than blindly suppressed? 38. Are seeds and failing sequences reproducible?

Gas, size and operations 39. What changed in deployment gas, hot-path gas, runtime/init bytecode, storage writes and maximum loop cost? 40. Does the contract retain size headroom on every target chain? 41. Can events reconstruct state and support reorg-safe indexers? 42. Are deployment values, code, proxy slots, roles, ownership, verification and post-deploy checks reproducible? 43. Are monitors and runbooks updated for new roles, events, values or failure modes?

Code quality 44. Are names precise about role, unit and lifecycle? 45. Are comments focused on invariants and why? 46. Are there dead paths, test-only production entry points, debug events, stale TODOs, unused imports or copied unsafe code? 47. Could a smaller public surface or clearer boundary remove an entire class of risk?

Conclusion. State the release decision; blockers and accepted risks; unverified assumptions; the exact commands and evidence reviewed; and the required follow-up owner and deadline.

PART D — Pull request template

```

## What changed
User-visible and protocol-visible behaviour. Affected contracts, interfaces, scripts, integrations.

## Why
Requirement, bug, threat, incident, optimisation baseline or governance decision.

## Assets and trust
- Assets / custody affected:
- Maximum value or blast radius:
- Trusted roles or dependencies changed:
- New external calls or callbacks:

## Invariants
- Introduced or changed:
- Why they still hold:
- Executable tests that enforce them:

## Compatibility
- ABI / selectors:          - Events / errors:
- Storage layout:          - CREATE2 / address assumptions:
- Off-chain indexer / backend / frontend:
- Breaking changes:

## Security analysis
- Reentrancy / callbacks:    - Authorisation / governance:
- Accounting / rounding:     - Oracle / MEV:
- Signatures / replay:       - Upgrade / cross-chain:
- Accepted risks and compensating controls:

## Verification
- Unit / regression:         - Fuzz:      - Invariant:
- Differential / model:       - Fork / integration:
- Static / symbolic analysis: - Coverage delta:

## Gas and size
- Compiler / EVM / optimizer / via-IR:
- Deployment gas delta:      - Critical-path gas delta:
- Runtime / init bytecode and headroom:
- Maximum batch / loop cost:

## Deployment and migration
- Networks:                  - Deterministic addresses / salts:
- Initializer / config values:
- Proxy / admin / implementation:
- Role / ownership transfer:
- Verification and smoke tests:
- Migration / order:          - Rollback or forward-fix plan:
- Monitoring / runbook changes:

## Reviewer focus
The highest-risk assumptions and the exact files and functions needing adversarial review.

## Checklist
- [ ] Scope is minimal and linked to a ticket or spec
- [ ] Compiler, dependency, ABI, event, error, storage and config diffs are declared
- [ ] Regression and invariant tests cover the change
- [ ] Analysis findings are triaged
- [ ] Gas / size and target-chain limits are checked
- [ ] Deployment / migration was rehearsed when applicable
- [ ] No secret, key, sensitive RPC value, debug output or unsafe artifact is committed

```